

重点回答：

- SGLang 的 RadixAttention / RadixCache 解决什么问题？

答：RadixCache(基数树 KV 缓存)，能够复用前缀 KV 缓存，避免重复的 prefill 计算，减少内存占用，从而提升推理吞吐量。

而传统的推理模式中，每条请求需要单独搞一套 KV，即便多条请求拥有相同前缀，也要重复 prefill，这就造成了内存的浪费。

- SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪

一部分？

答：page-sized KV cache（分页 KV 缓存），它是底层 KV 内存管理机制，其作用于 Prefill 阶段的结果存储（得到新的 token 的 KV）与 Decode 循环之间，将 KV 切分成固定大小的内存页，能够支持按需分配、释放页面，避免了连续大内存块带来的碎片问题；

Prefix reuse(前缀复用)，它是 RadixCache 的核心逻辑，发生在调度器接受请求并把请求列入队列之后，正式开始 prefill 计算之前；调度器会查询基数树，尝试匹配之前的所有 token(即 prompt)已存在的前缀 KV，如果匹配成功就复用已有 KV，就不用在 prefill 阶段重新计算全部输入 prompt，直接继续预测下一个 token 就行，而只对新出现的 token 要进入 prefill 阶段..

- vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention

/ paged KV 机制有什么联系与差异？

答：PagedAttention 解决了大模型推理过程中存在的 KV 缓存的内存碎片化、内存分配低效的问题，其借鉴了操作系统虚拟内存分页思想，把 KV 缓存拆成了一个离散的、固定大小的内存页，这样，页面不需要连续了，能够支持动态申请与释放 KV 页，显著地提升了显存利用率与服务器吞吐量；

二者的联系：它们都采用了分页 KV 存储，把 KV 切分为了独立内存页，且能够适配连续批量处理，都能提升推理服务吞吐量；

二者的差别：PagedAttention 只实现了 KV 缓存分页内存管理，不支持共享 KV 缓存，各个请求的 KV 页面相互独立，没有前缀复用能力；而 RadixAttention 在分页 KV 的基础上叠加了基数树索引结构，能够支持共享 KV，但这也有代价，它引入了基数树查找的开销，这也会消耗算力，只是在请求前缀重合度高的情况下，这个机制能够明显节省算力。